Where the Learned Dissipativity Conditions Hold, and a Change That Makes Them Hold Almost Everywhere: All-VSG Kundur

Sehaj Randhir Singh · October 2026 · built on the all-VSG Kundur example in Wang, Wang, Zhuang, Moffat and Dörfler, arXiv:2510.22324

Do the learned conditions hold along the closed loop, where Theorem 2 uses them, and not only at the training tuples? In my rebuild of the example they hold at every training tuple and fail on most of the region once $u = \pi(x)$: the dissipativity inequality is violated at 84% of sampled states and at 100% near the equilibrium. The reason turned out to be specific and fixable. Near the equilibrium the inequality asks the controller for about 27 times more decrease in V per step than a 0.5 ms step can deliver. Building $\Phi \succ 0$ into the parametrization and training the inequality at $u = \pi(x)$ brings the violation down to 10% of the region, the closed-loop decrease violation from 47% to 7%, and $\Phi \succ 0$ holds at every sampled state.

Why the training tuples can't see this

The inequality is trained on tuples where u is drawn uniformly over the input range. At those tuples the supply rate s(x, u) is large, with a median of about 85 against a storage change of about 0.006 per step, so the inequality is easy to satisfy. Theorem 2 consumes the closed loop, and at $u = \pi(x)$ the supply rate falls to its minimum, $-x^T\Phi(x)x$. That input is exactly where the inequality has the least slack, and a uniform draw almost never lands on it, so a zero training loss says little about the case the proof needs.

Rebuilding the example

I rebuilt the Kundur two-area system with all four generators as VSGs using the paper's parameters (virtual impedance j0.3, H = 4 s, damping 5 pu on the 900 MVA base) as a Kron-reduced second-order swing model with a 0.5 ms zero-order-hold step and one B8-B9 circuit open after the fault. Without control it loses synchronism after the fault as in the paper's Fig. 6. The controller follows the paper's parametrization, matrix networks W(x), Q(x), S(x) and $R^{-1}(x)$ with $\pi(x) = -R^{-1}(x)S(x)^Tx$, trained on the paper's losses on 2,500 one-second trajectories (98,069 tuples). The region check samples 200,000 states over the paper's region X ($|\Delta\delta| \le 2\pi$, $|\Delta\omega| \le 0.1$ pu) and 200,000 each in boxes at 5% and 1% of X, and runs multi-start gradient attacks.

What the failure is

Comparing the two sides of the inequality state by state shows what goes wrong. In the boxes at 1%, 3% and 10% of X, the decrease the inequality demands, $x^T\Phi x$, is a median 27.5 times the decrease $V(x) - V(x^+)$ that the closed loop actually produces in one step, and 39 times over the whole region. The plant can't close that gap in 0.5 ms, since with H = 4 s the state barely moves per step and V drops by about 10^{-5} of its value. So the inequality isn't failing because the controller is bad at stabilizing. It's failing because Φ, which sets the required decrease, was learned only to be positive definite and not small enough to match one sampling step.

A first attempt shows why Φ has to be handled structurally. Training the inequality at $u = \pi(x)$ alone drives its violation to 0.01% of the region within one epoch, but it gets there by making Φ indefinite at every sampled state, which voids the theorem. Adding a $\Phi \succ 0$ penalty didn't stop that within four epochs, because the two terms pull against each other.

The change

I kept every network and reinterpreted the Q head. It now outputs a positive definite matrix P(x), and $Q := S\,R^{-1}\,S^T - P$, so $\Phi = S\,R^{-1}\,S^T - Q = P(x) \succ 0$ holds by construction instead of by penalty. I then warm-started from the

reproduced controller and trained for 26 epochs with the original losses plus the inequality and the closed-loop decrease evaluated at u = π(x), on states drawn half over X and half in boxes at 10%, 3% and 1% of X around the equilibrium. After training, the required decrease is a median 0.10 of the decrease each step delivers, so the scale mismatch is gone.

Result

Condition, share of sampled states violating	Reproduced controller	Restructured controller
Dissipativity inequality at u = π(x), whole X	84%	10%
Same, box at 1% of X	100%	16%
Closed-loop decrease $V(x^+) \le V(x)$, whole X	47%	7%
Same, box at 1% of X	50%	11%
$\Phi(x) \succ 0$, whole X	23% not positive definite	0% (by construction)
Input range $\lvert\pi(x)\rvert \le 0.3$ pu	holds	holds
Required over delivered decrease near equilibrium (median)	27.5×	0.10×

Dense samples of 200,000 states per region, with the training margin ε set to 0 in the inequality (see Limits). Numbers are from results/recheck_eps0_*.json and results/diag_*.json.

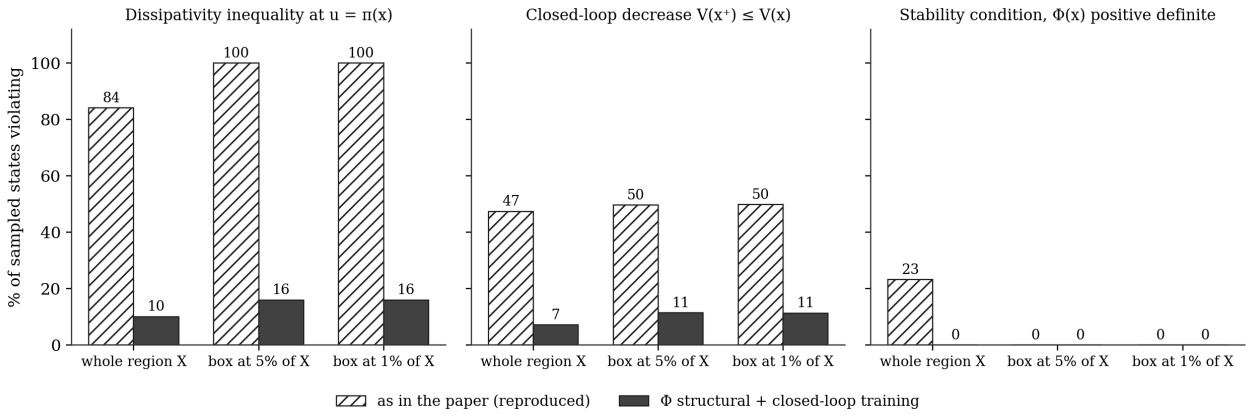


Figure 1. Share of sampled states violating each condition, for the reproduced controller and the restructured one, over the whole region and in the boxes around the equilibrium.

What it means

The interesting part, to me, is that the conditions fail for a reason you can measure and design around. The inequality ties the required decrease to Φ, and in discrete time the decrease available per step is set by the sampling time and the inertia, so a Φ that is only asked to be positive definite can be orders of magnitude too large for the plant to honor. Making Φ positive definite by construction removes the conflict between the two losses, and training at u = π(x) puts the inequality where the theorem reads it. I think the same mismatch would show up in other learned dissipativity controllers with fast sampling and slow plants.

The open piece is the fault ride-through. With $\lvert u\rvert \le 0.3$ pu, the trajectory after the paper's fault is nearly the same with no control, the reproduced controller or the restructured one, and it leaves X at about 1.88 s. That suggests the input bound I chose, and not the conditions, decides that case, and I'd value your read on the input scaling you used.

Limits, stated flatly

This is my rebuild with my own choices where the paper doesn't fix them (epsilons, cost-shaping form, mean-plus-max aggregation, an input-range penalty), not the authors' controller. All numbers are dense-sample searches and gradient attacks, which find violations but don't prove their absence, and the remaining 10% to 16% are real violations the attacks find. I couldn't run a sound CROWN certificate because auto_LiRPA 0.7.2 doesn't support the GELU and softplus activations the networks use. The inequality is checked with $\varepsilon = 0$ here, because a constant margin of 10^{-3} can't hold as $x \to 0$ when both sides are around 10^{-6}, which also means the near-equilibrium numbers in my first draft overstated the failure. Current limiting is not modeled. I'm happy to share the code and the trained controllers.